

App design → 12

N/W Essentials - 1

DB indexing - 2

API Gateway - 3

CAP theorem - 4

Consistent Hashing - 5

Caching → 6

Big ds → 11

Object storage → 8

Sharding → 7

Message queues → 13

distributed transactions → 10

Realtime collaborations → 22

Concurrency → 16

data modeling → 9

Deep-dives

7

Redis, RabbitMQ vs Kafka, Kafka, 14 15

dynamodb - 17
cassandra - 18

Elastic search - 19

Time Series db - 20

Proximity Search - 21

bitly
distributed Rate Limiter → 12

E-commerce

Bot detection (ML) f

Amazon locker U V

Connect, top-K t s

Fb post search g

Yelp, online Auction R

News feed, Youtube e

fb live comments (j)
whatsapp (k)
tinder (l)
leetcode (m)
web crawler (n)
Ad-click aggregator (o)

dropbox (h)
uber (i)
ticketmaster (a)
twitter (p)

SD → (1) (2)
SD-for AI

AWS/Azure/GCP

hello interview
(2) (ud)

Docker
Kubernetes

N/W essentials → 18/09/2026

N/W 101

L-3 protocols → IP

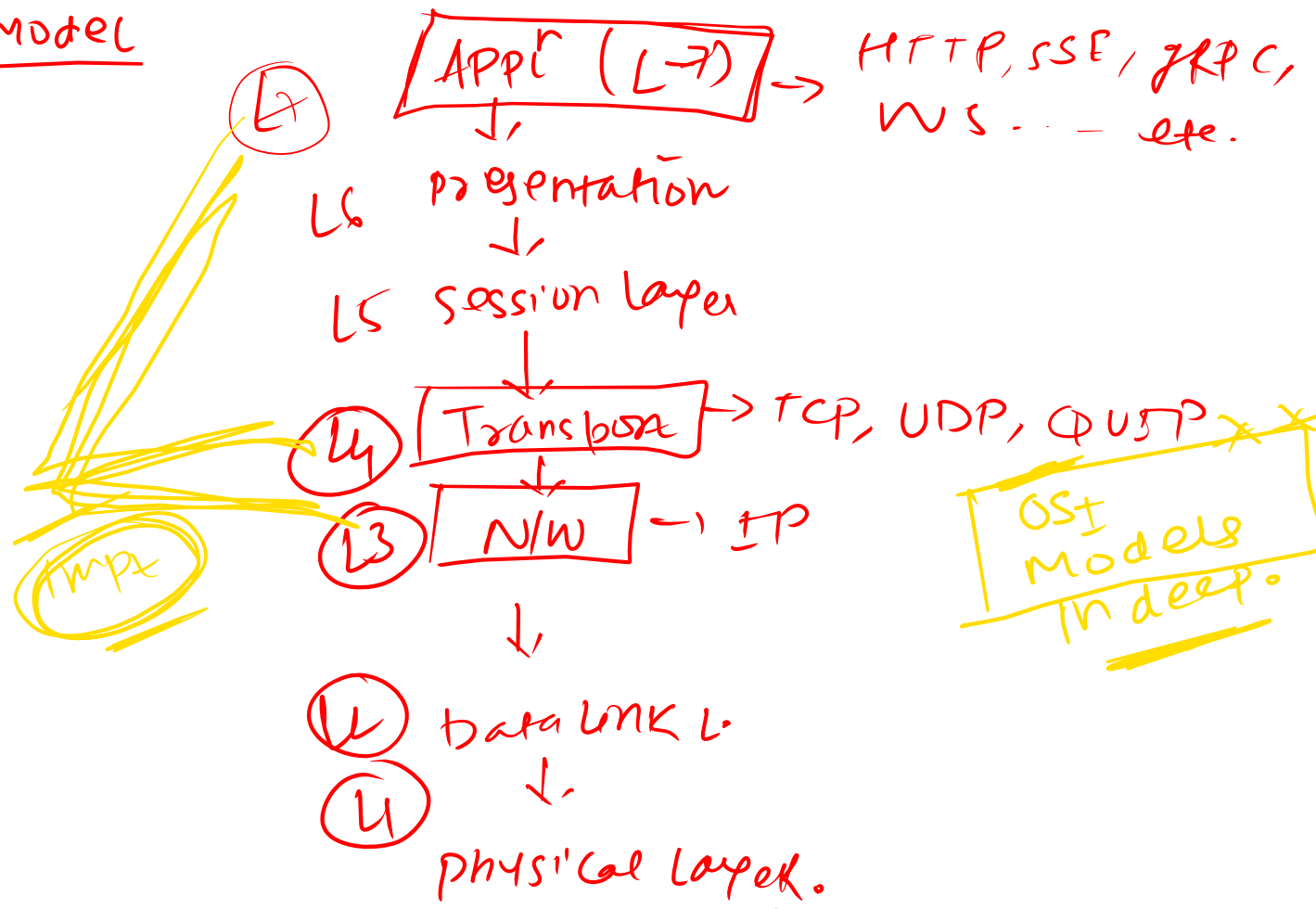
L-4 protocols → TCP, UDP, QUIC

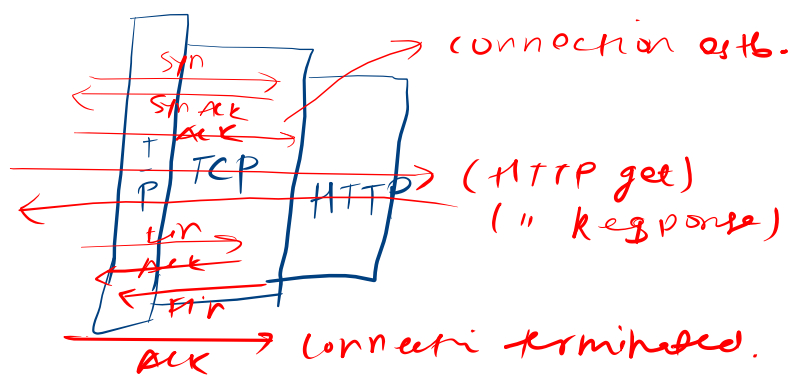
L-7 protocols → HTTP, REST, gRPC, SSE, WS, webRTC

Load balancing

Deep dives → Regionalization, timeouts and Retries, cascading failures.

OSI Model





Back & forth
 (latency)
 + state
 ↓
connection ✓
 ✗

(IP) Internet protocol.

IPv4 (4 bytes) externally

internally
 IPv6 (16 bytes)

50:06:226.113 ⇒ Run out

⇒ Private Address

192.168.0.0/16

Internet host

⇒ Public Address

[18.0.0.0/8]

public

Apple

LB
 App Gateway

Layer 4 protocols

TCP

1 2 3 4 5
 data packets
 210s+
 You can wait & ask for again

- # Connection-oriented
- # Reliable delivery
- # Guaranteed Ord.
- # Higher Latency 😞

x flow control, congestion control

x default, reliable

re-transmission

UDP ordering and delivered or not

connectionless

NO delivery Guarantee

NO ordering

Lower Latency

(Real time comm)
Machine gun

Q VEC

TCP subs.

context → source destination → PORT

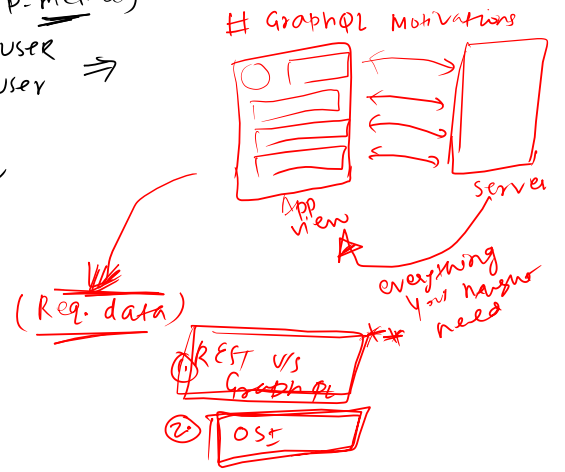
21-7) protocols ① HTTP Example

Request Method/verb Intent
GET /posts/HTTP/1.1
host:
Accept: application/json
User-Agent → (Mozilla/5.0 (Windows 10))
headers (K=V)

Response status code
HTTP/1.1 200 OK
Date: -
Content-type: -
Content-length: -
headers Body

REST API (HTTP method)

GET /user/{id} → user
POST /users → user
PUT /user/{id} → update
DELETE /user/{id} → delete



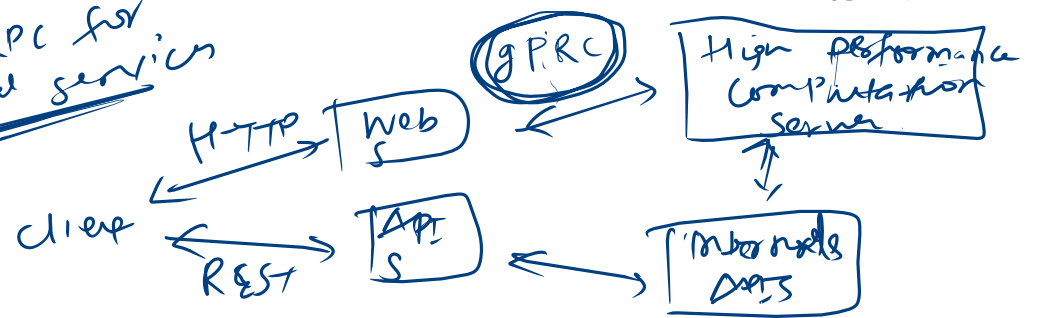
gRPC (google Remote Procedure Calls)

↳ protobuf + Service Message work
schema (struct in C++)
Serialized and easy/optimized design
10x throughput (REST)
str id;
str name;
15 byte → protobuf representation

x client side
LB, streaming
etc.

external client (web browser)
↓
doesn't support gRPC
devs (binary)
↳ server binary

Using gRPC for internal services



SSE (server sent events)

Notification

delayed by polling
freq

Overhead
→ Repe.

Open and
Close connection
to show
Req

extension^{on} HTTP Req.

variable
one del~~st~~

Response: - "200"

Unidirectional
(no ex
info)

client side

30-1 sec Timeout

Long Running.
Req. server can
push events down
so client

SSE

websockets

high freq. updates and
bi-directional comm

Require a lot of
infra. ^{SD}

think polling based solⁿ, SSE

exchange binary blobs

websockets

scaling

lot of states

~~stateful~~

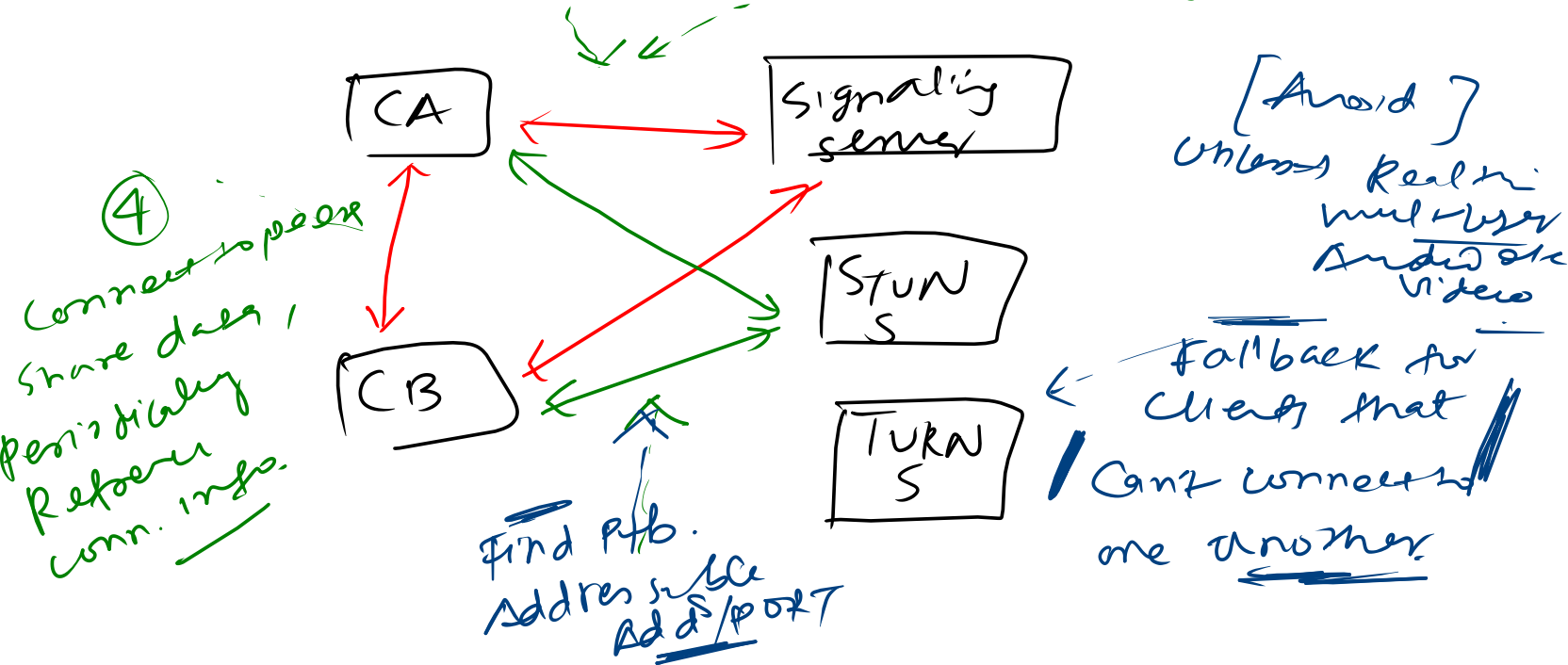
WebRTC

Runs over UDP.

* collaborative editors, Audio and video
Chats

peer-to-peer connections

- ① connect to SS and learns abt peers
- ② share connection info b/w peers



Deep dives

Regionalizations

Applⁿ across globe

and N/W latency
fixed (80ms)

Global Scale
office

Fiber
cables + light & speed



[Uber → same city]

CDN

Data centers

Replication
lag (sync)
CDN (cache)

edge location

partition
data and
keep close to
users as possible

Timeouts, Retries, Backoffs

N/W ⇒ goes down

Timeouts : → Retries

(5sec) --- (5sec) ---

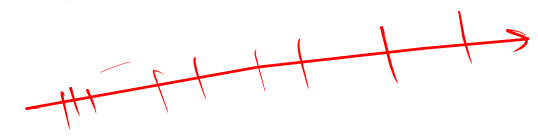
Retries 2KHz + handle



Backoff
exp.

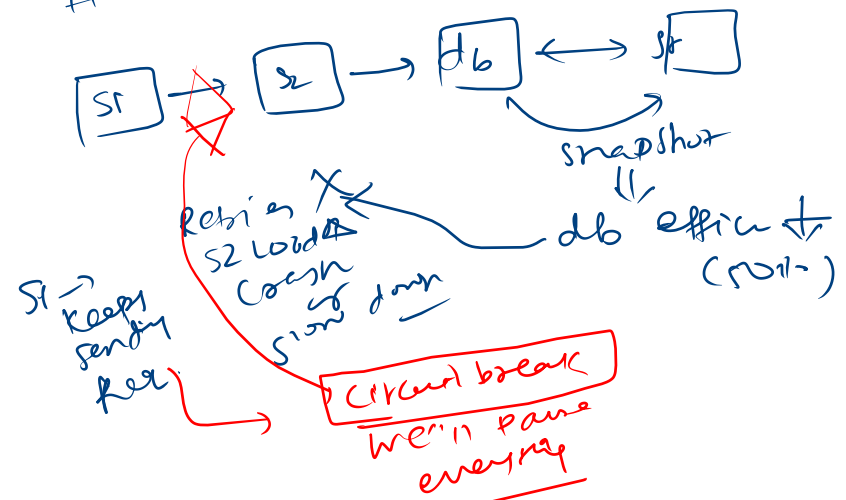


→ Jitter



cascading failure

Circuit Breaker



DB indexing



Find a Row in a db without index?

1. Pull a page into memory and search for Row.
2. Pull another page ---
3. Repeat ---

page → 100 rows
100 users → 1M pages

Round trip SSD to Hard drive
100 μs
1000000 × 100 μs = 100 seconds
⇒ 3-5 seconds

Indexes → ds
act as on map and tell where that items lie

Index → then only few pages

Range query

1. B-tree

